```
    }catch (ReflectionException $e){
      $this->strategy = "";
    }
  }
  function goToSchool(){
    $this->strategy->wayToSchool();
  }
}
// Testing
$context  = new Context();
$context->getStrategy('BusStrategy');
$context->goToSchool();
```

***Operation results***

Take a bus to school

## *Advantage*

- The policy pattern provides a way to manage the related algorithmic families. The hierarchical structure of policy classes defines an algorithm or behavior family. Proper use of inheritance can transfer common code to the parent class, thus avoiding duplication of code.
- Policy patterns provide a way to replace inheritance relationships. Inheritance can handle a variety of algorithms or behaviors. If not using policy patterns, then there may be some subclasses of environment classes that use algorithms or behaviors, each of which provides different algorithms or behaviors. However, the user of the algorithm or behavior is mixed with the algorithm or the behavior itself. The logic of deciding which algorithm to use or which behavior to take is mixed with the logic of the algorithm or behavior, so that it is impossible to evolve independently. Inheritance makes it impossible to dynamically change an algorithm or behavior.
- Multiple conditional transfer statements can be avoided by using policy patterns. Multiple transfer statements are not easy to maintain. It combines the logic of which algorithm to adopt or which behavior to adopt with the logic of algorithm or behavior, and lists them all in a multiple transfer statement, which is more primitive and backward than the method of inheritance.